

Exercise Sheet 8: Energy-based Models and Score Matching

Due on 03.07.2026, 10:00

Jannik Wiese (jannik.wiese@lmu.de)

Important Notes:

1. **Questions:** Please use the Moodle forum for questions. They will be answered by us or your fellow students.
2. **Deadline:** The due date for this exercise is **03.07.2026, 10:00**.
3. **Submission:** Submit one ZIP file containing your completed notebook and a single PDF report. Both files should contain your surname and matriculation number, e.g. *Surname-MatriculationNumber.zip*. The exported PDF should show your code, plots, and printed metrics. **Submissions that fail to follow the naming convention or miss the PDF/notebook files will not be graded.**

Use the notebook `exercise8.ipynb`. Complete all # TODO cells and keep the requested plots and printed metrics visible in your submitted notebook/PDF.

Task 1: Sampling from Energy Models (5P)

Background. We can write the probability of an image $\mathbf{x}$ as $p(\mathbf{x}) = \frac{\exp(-E(\mathbf{x}))}{Z}$ where $Z = \int_{\mathbf{x}} \exp(-E(\mathbf{x}))$ is a (intractable) normalizing constant. Nevertheless, we can learn the energy function $E_{\theta}(\mathbf{x})$ using a neural network. This allows us to compute the score $\nabla_{\mathbf{x}} \log p_{\theta}(\mathbf{x}) = -\nabla_{\mathbf{x}} E_{\theta}(\mathbf{x})$ pointing in the direction of more likely samples. Using the score we move our current estimate $\mathbf{x}_t$ towards a higher probability estimate $\mathbf{x}_{t+1}$ by using Langevin dynamics:

$$\mathbf{x}_{t+1} = \mathbf{x}_t - \frac{\alpha^2}{2} \nabla_{\mathbf{x}} E_{\theta}(\mathbf{x}_t) + \alpha \mathbf{z}, \quad \mathbf{z} \sim \mathcal{N}(0, I) \quad (1)$$

where α is the step size.

Task. For this task consider the (two dimensional) energy

$$E(x, y) = -\log \left(e^{-\frac{\|(x, y) - (-2, 0)\|^2}{2}} + e^{-\frac{\|(x, y) - (2, 0)\|^2}{2}} \right). \quad (2)$$

The corresponding density is

$$p(x, y) = \frac{e^{-d_1^2/2} + e^{-d_2^2/2}}{4\pi}. \quad (3)$$

with

$$d_1^2 = \|(x, y) - (-2, 0)\|^2, \quad d_2^2 = \|(x, y) - (2, 0)\|^2. \quad (4)$$

1. (2P) Implement sampling via Langevin dynamics. Start from 500 points $\mathbf{x}_0 \sim \mathcal{N}(0, I)$ and sample 500 data estimates by using 700 Langevin updates with step size 0.1. Use torch for your implementation.
2. (1P) Plot the obtained samples twice: once over the energy contours $E(x, y)$ and once over the density $p(x, y)$. Note down your observations.
3. (1P) Run an ablation experiment comparing different sampling steps ($N \in [1, 10, 100, 1000]$) and step sizes ($\alpha \in [0.001, 0.01, 0.1, 1]$) (16 runs in total). Plot samples for each combination. Note down your observations, what effect do the parameters have?
4. (1P) Explain why samples accumulate in low energy regions through Langevin sampling. Further, explain why we need to add noise during sampling.

Task 2: Training an EBM denoiser (8P)

Background. For more complex data, we have to learn the energy model $E_\theta(\mathbf{x})$ from data. In a perfect world we want our model to optimize $\max_\theta \mathbb{E}_{\mathbf{x} \sim p_{data}} [\log p_\theta(\mathbf{x})]$ but the ground truth distribution is unknown. We can reformulate to obtain

$$\nabla_\theta \log p_\theta(\mathbf{x}) = -\nabla_\theta E_\theta(\mathbf{x}) - \nabla_\theta \log Z(\theta). \quad (5)$$

Through some algebra, the second term can be rewritten as

$$\nabla_\theta \log Z(\theta) = \mathbb{E}_{\mathbf{x} \sim p_\theta} [-\nabla_\theta E_\theta(\mathbf{x})]. \quad (6)$$

This is convenient because we have already seen how we can sample from p_θ . Therefore, maximizing the log-likelihood consists of two competing objectives:: 1) decrease energy of real samples, 2) increase energy of samples produced by Langevin dynamics with the current model. Putting the loss together we get

$$\nabla_\theta \log p_\theta(\mathbf{x}) \approx -\nabla_\theta E_\theta(\mathbf{x}) + \nabla_\theta E_\theta(\tilde{\mathbf{x}}), \quad \tilde{\mathbf{x}} \sim p_\theta. \quad (7)$$

Because sampling good $\tilde{\mathbf{x}}$ from scratch is slow, contrastive divergence¹ instead proposes to initialize Langevin dynamics from a noisy version of a real data sample and perform only a small number of Langevin updates.

Task.

1. (3P) Implement an energy model consisting of a 3-layer CNN with Max-Pooling followed by a 2-layer MLP that outputs a scalar energy. Then implement sampling via Langevin dynamics for this model by computing the score via automatic differentiation using `torch.autograd.grad(energy, x)[0]`. The sampler should start from an initial (noisy) image and perform several Langevin updates while keeping pixel values in the valid range.
2. (3P) Train the energy model on the MNIST dataset using contrastive divergence. Initialize the negative samples with a noisy version of each training image ($\sigma^2 = 0.3$) and refine them using the Langevin sampler from part (a). Perform 30 sampling steps with $\alpha = 0.2$ and 0.01 noise scale. Optimize the contrastive divergence objective with the Adam optimizer for 5000 training steps. Add an L_2 regularization term on the predicted energies to prevent the energy values from diverging. Visualize the provided real, noisy and denoised samples every 1000 training steps.

¹Hinton, Geoffrey E. "Training products of experts by minimizing contrastive divergence." Neural computation 14.8 (2002): 1771-1800. <https://www.cs.toronto.edu/~hinton/absps/tr00-004.pdf>

3. (2P) Evaluate your trained EBM using the relative log-probability equation from the lecture. Compare the average relative log probabilities and average relative probabilities of (1) random noise, (2) noisy images, (3) denoised images obtained with Langevin dynamics, and (4) ground truth images over 1000 samples. Note down your observations and explain why machine learning papers typically report log probabilities (or log-likelihoods) instead of probabilities.

Task 3: Score Matching (7P)

Background. In Energy-based Modeling we approximate the score $\nabla_{\mathbf{x}} \log p(\mathbf{x})$ using gradients on the unnormalized energy. Score matching proposes to instead learn the score $s_{\theta}(\mathbf{x}) \approx \nabla_{\mathbf{x}} \log p(\mathbf{x})$ directly. However, the true score $\nabla_{\mathbf{x}} \log p(\mathbf{x})$ is unknown, making direct supervision tricky. Denoising Score Matching presents an alternative where we learn to instead approximate the conditional score $s_{\theta}(\mathbf{x}_t) = \nabla_{\mathbf{x}_t} \log q(\mathbf{x}_t|\mathbf{x}, \sigma_t^2 I)$ where $q(\mathbf{x}_t|\mathbf{x}, \sigma_t^2 I)$ is a Gaussian distribution centered at $\mathbf{x}$ with variance σ^2 . Given a data sample $\mathbf{x}$ we can sample from the conditional distribution $q(\mathbf{x}_t|\mathbf{x}, \sigma_t^2 I) = \mathcal{N}(\mathbf{x}_t|\mathbf{x}, \sigma_t^2 I)$ using the reparametrization trick:

$$\mathbf{x}_t = \mathbf{x} + \sigma_t \epsilon \sim \mathcal{N}(\mathbf{x}_t|\mathbf{x}, \sigma_t^2 I), \quad \epsilon \sim \mathcal{N}(0, I). \quad (8)$$

Given $\mathbf{x}$ the conditional score has a simple closed form solution

$$\nabla_{\mathbf{x}_t} \log \mathcal{N}(\mathbf{x}_t|\mathbf{x}, \sigma_t^2 I) = \frac{\mathbf{x} - \mathbf{x}_t}{\sigma_t^2} \quad (9)$$

which we can use to supervise $s_{\theta}(\mathbf{x}_t, \sigma_t)$.

Task.

1. (2P) Mathematically derive the closed-form conditional score in Eq. 9 starting from the definition of $q(\mathbf{x}_t|\mathbf{x}, \sigma_t^2 I)$. Give all your steps, as they will be considered for grading!
2. (4P) Train a denoising score model. Implement $s_{\theta}(\mathbf{x}_t, \sigma_t)$ as a small convolutional U-Net² (most of the implementation is already provided but you have to fill in some parts of the forward pass) and train it to predict the score of noisy MNIST images using an Adam optimizer for at least 10k iterations (better 50k). During training, randomly sample the noising level σ_t from a uniform distribution $\sigma_t \sim \mathcal{U}(0.01, 2)$. Normalize your MNIST images to $[-1, 1]$ range before applying noise (don't forget to unnormalize samples for

²<https://medium.com/@alejandro.itoaramendia/decoding-the-u-net-a-complete-guide-810b1c6d56d8>

visualization). Condition your model on the current σ_t -level. Use the provided sigma embedding and learn a projection of that embedding. For each Unet resolution, add the projected sigma embedding to the Unet features. Train the model with an MSE objective between the model prediction and the ground truth conditional score derived above. Weight the MSE by the current noising level σ_t^2 .

3. (1P) Sample 10 generated images from your model by implementing a third version of Langevin dynamics starting from Gaussian noise and using the predicted score. Use 50 decreasing and equally spaced σ -levels on each σ -level perform 20 Langevin sampling steps. For each Langevin update use stepsize $\alpha = \beta * (\sigma/\sigma_{min})^2$ where $\beta = 0.00002$. Visualize your samples.